

10 PROJECT OPTIONS: ML | DL | CV | NLP | RL | GENAI | LLM FINE-TUNING | RAG | CODING AGENT | AI EVALS

01

MACHINE LEARNING / SKILLSENS

Predict placement readiness and explain the factors behind the result.

ONE-MONTH CHALLENGE: Choose exactly one of the ten projects. You have 30 days to build a working system, document your decisions and show evidence that it works. AI tools are allowed, but you must understand and be able to explain everything you submit.

WHAT YOU WILL BUILD

Build a machine-learning web app that estimates a student's placement readiness from tabular data such as academics, skills, projects, coding practice and internships. The app should return a readiness score or category and clearly show the main factors behind the result.

CORE REQUIREMENTS

- Prepare or curate a tabular dataset and define the prediction target clearly.
- Build a repeatable cleaning and preprocessing pipeline for missing values, categorical fields and numeric features.
- Train a simple baseline such as logistic regression or a decision tree, then compare it with at least one stronger model.
- Use a fixed train/validation/test split and check that target leakage is not present.
- Show a readiness score or category together with feature importance or key positive and negative factors.
- Support single-student input plus CSV upload for batch predictions.
- Build a clean dashboard with summary cards, charts, filters and a small what-if view where changing inputs updates the prediction.

WHAT MUST BE YOUR OWN WORK

- The preprocessing and feature-engineering pipeline.
- The model-comparison and evaluation runner.
- The explanation and what-if logic shown in the app.

30-DAY BUILD PLAN

Week 1 - define the dataset and target, clean the data and build the baseline. Week 2 - train stronger models, tune them and freeze the test set. Week 3 - add explainability, CSV predictions, charts and failure analysis. Week 4 - finish the web app, tests, documentation and demo.

EVIDENCE YOU MUST SHOW

- Report precision, recall, F1 and a confusion matrix; add ROC-AUC when the task is binary.
- Compare at least two models on exactly the same test set.
- Show global feature importance and explanations for several individual predictions.
- Test missing values, unseen categories and noisy inputs without crashing.
- Analyse at least 20 wrong or uncertain predictions and explain likely causes.

FINAL SUBMISSION PACKAGE

- Working repository with meaningful commit history.
- README with exact setup and run commands.
- Architecture diagram and important design decisions.
- Automated tests plus a written failure log.
- 5-8 minute demo video and a concise final report.
- AI_USAGE.md describing how AI tools were used and verified.

SUGGESTED TOOLS

Python, pandas, scikit-learn, optional XGBoost/LightGBM, Streamlit or React.

HIGH BAR: A notebook with one model and one accuracy score is not a finished SkillLens submission.

10 PROJECT OPTIONS: ML | DL | CV | NLP | RL | GENAI | LLM FINE-TUNING | RAG | CODING AGENT | AI EVALS

02

DEEP LEARNING / ECOSORT AI

Classify waste images into dry, wet, recyclable and e-waste.

ONE-MONTH CHALLENGE: Choose exactly one of the ten projects. You have 30 days to build a working system, document your decisions and show evidence that it works. AI tools are allowed, but you must understand and be able to explain everything you submit.

WHAT YOU WILL BUILD

Build a deep-learning image-classification app that looks at a waste photo and predicts whether it is dry, wet, recyclable or e-waste. Use transfer learning, compare model choices, show confidence, and create an interface that works with uploaded images and, where possible, a webcam.

CORE REQUIREMENTS

- Collect or prepare a labelled image dataset for dry, wet, recyclable and e-waste, and document where the images came from.
- Create separate train, validation and test splits without duplicate-image leakage.
- Train a baseline model and at least one transfer-learning model; compare two model choices fairly.
- Use useful augmentation such as crop, flip, rotation or brightness changes and measure whether it helps.
- Track training/validation loss and accuracy, save the best checkpoint and make runs reproducible.
- Return the predicted class, confidence score and top alternatives for every image.
- Add an image upload or webcam demo plus a simple Grad-CAM heatmap that highlights what influenced the prediction.

WHAT MUST BE YOUR OWN WORK

- The dataset preparation, split and augmentation pipeline.
- The training/experiment loop and model-comparison report.
- The error-analysis and visual-explanation flow.

30-DAY BUILD PLAN

Week 1 - prepare the dataset, splits and a working baseline. Week 2 - train transfer-learning models and run augmentation experiments. Week 3 - compare models, inspect confusion cases and add Grad-CAM. Week 4 - build the upload/webcam app, tests, documentation and demo.

EVIDENCE YOU MUST SHOW

- Report per-class precision, recall, F1 and a confusion matrix - not accuracy alone.
- Compare two model choices on the same frozen test set.
- Test real phone photos with different lighting, backgrounds, angles and partially visible objects.
- Show at least 20 misclassified images and explain the likely reason for each failure type.
- Show the effect of augmentation and report practical inference speed for the final model.

FINAL SUBMISSION PACKAGE

- Working repository with meaningful commit history.
- README with exact setup and run commands.
- Architecture diagram and important design decisions.
- Automated tests plus a written failure log.
- 5-8 minute demo video and a concise final report.
- AI_USAGE.md describing how AI tools were used and verified.

SUGGESTED TOOLS

Python, PyTorch or TensorFlow, OpenCV, torchvision/timm, Streamlit or Gradio.

HIGH BAR: Using one pretrained model without training, comparison and error analysis is not a complete EcoSort AI project.

10 PROJECT OPTIONS: ML | DL | CV | NLP | RL | GENAI | LLM FINE-TUNING | RAG | CODING AGENT | AI EVALS

03

COMPUTER VISION / TRACKFORGE

Detecting an object is easy. Keeping its identity is the real challenge.

ONE-MONTH CHALLENGE: Choose exactly one of the ten projects. You have 30 days to build a working system, document your decisions and show evidence that it works. AI tools are allowed, but you must understand and be able to explain everything you submit.

WHAT YOU WILL BUILD

Build a real-time video system that detects, tracks and uniquely counts objects across frames. You may use a pretrained detector, but the tracking, object lifecycle and event logic must be your own. The final application should work on uploaded videos and, where possible, a live camera.

CORE REQUIREMENTS

- Run object detection and convert detections into a clean internal format.
- Implement a baseline tracker using centroid distance or IoU matching.
- Improve association using Hungarian matching and optional motion prediction.
- Maintain stable track IDs and handle new, missing, lost and recovered tracks.
- Support line crossing, entry/exit, polygon zones, dwell time and direction.
- Prevent double counting when an object pauses, disappears or re-enters.
- Store event history with timestamps, class, track ID and confidence.
- Build a UI that shows video, tracks, paths, zones, counts and live FPS.

WHAT MUST BE YOUR OWN WORK

- Detection-to-track association.
- Track lifecycle and occlusion handling.
- Zone, crossing and unique-count event logic.

30-DAY BUILD PLAN

Week 1 - detector pipeline, annotation format and baseline tracker.

Week 2 - robust association, track lifecycle and event engine.

Week 3 - ground-truth videos, metrics and difficult-case testing.

Week 4 - interface, logging, optimisation, documentation and final demo.

EVIDENCE YOU MUST SHOW

- Manually annotate at least three short videos as ground truth.
- Measure counting error, duplicate counts, missed crossings, ID switches and FPS.
- Test crowded scenes, temporary occlusion, low light and objects leaving/re-entering.
- Compare at least two association strategies on the same videos.
- Show failure clips and explain why the tracker broke.

FINAL SUBMISSION PACKAGE

- Working repository with meaningful commit history.
- README with exact setup and run commands.
- Architecture diagram and important design decisions.
- Automated tests plus a written failure log.
- 5-8 minute demo video and a concise final report.
- AI_USAGE.md describing how AI tools were used and verified.

SUGGESTED TOOLS

Python, OpenCV, NumPy, Ultralytics/PyTorch, Streamlit/Gradio/React. The tracker cannot be a one-line library call.

HIGH BAR: Drawing detector boxes and incrementing a counter once per frame is not object tracking.

10 PROJECT OPTIONS: ML | DL | CV | NLP | RL | GENAI | LLM FINE-TUNING | RAG | CODING AGENT | AI EVALS

04

NATURAL LANGUAGE PROCESSING / CLARITYROUTE

Turn messy human messages into useful, reviewable decisions.

ONE-MONTH CHALLENGE: Choose exactly one of the ten projects. You have 30 days to build a working system, document your decisions and show evidence that it works. AI tools are allowed, but you must understand and be able to explain everything you submit.

WHAT YOU WILL BUILD

Build a support-message intelligence system that predicts intent, urgency and important entities. Real messages may contain spelling errors, several intents, short context or completely unknown topics. Your system must recognise uncertainty instead of forcing every message into a known class.

CORE REQUIREMENTS

- Create an annotation guide and curate a clean labelled dataset with several intents.
- Build a TF-IDF plus linear-model baseline before using a transformer.
- Support multi-label intent classification when one message has multiple requests.
- Extract useful entities such as product, department, date, location or priority.
- Fine-tune or adapt a compact transformer and compare it with the baseline.
- Calibrate confidence and add an unknown-topic/abstention decision.
- Expose a /predict API that returns labels, entities, confidence and warnings.
- Build a review screen where uncertain or wrong predictions can be corrected.

WHAT MUST BE YOUR OWN WORK

- The data/annotation schema and validation pipeline.
- Multi-label threshold selection and abstention logic.
- The unified evaluator for labels, entities and confidence.

30-DAY BUILD PLAN

Week 1 - annotation guide, data cleaning and TF-IDF baseline.

Week 2 - transformer, multi-label support and entity extraction.

Week 3 - confidence, unknown-topic detection and adversarial tests.

Week 4 - API, review UI, documentation and full comparison report.

EVIDENCE YOU MUST SHOW

- Report macro-F1, micro-F1, per-class recall, entity F1 and calibration.
- Create stress tests for spelling mistakes, paraphrases, mixed intents and unrelated text.
- Analyse at least 30 errors and group them into meaningful failure types.
- Compare the baseline and transformer on the same frozen test set.
- Show how corrections could be collected for future retraining.

FINAL SUBMISSION PACKAGE

- Working repository with meaningful commit history.
- README with exact setup and run commands.
- Architecture diagram and important design decisions.
- Automated tests plus a written failure log.
- 5-8 minute demo video and a concise final report.
- AI_USAGE.md describing how AI tools were used and verified.

SUGGESTED TOOLS

Python, scikit-learn, Hugging Face Transformers, FastAPI and any suitable frontend.

HIGH BAR: A two-label sentiment model with one /predict endpoint is below the required scope.

10 PROJECT OPTIONS: ML | DL | CV | NLP | RL | GENAI | LLM FINE-TUNING | RAG | CODING AGENT | AI EVALS

05

REINFORCEMENT LEARNING / **QUEUECRAFT**

Build the world, the baselines and the agent - then prove the agent helps.

ONE-MONTH CHALLENGE: Choose exactly one of the ten projects. You have 30 days to build a working system, document your decisions and show evidence that it works. AI tools are allowed, but you must understand and be able to explain everything you submit.

WHAT YOU WILL BUILD

Build a scheduling simulation in which jobs arrive with different durations, priorities and deadlines. An agent must decide which job should run on which limited resource. The project is as much about environment and reward design as it is about the learning algorithm.

CORE REQUIREMENTS

- Define state, action, transition and reward clearly before coding.
- Build a custom environment with `reset()`, `step()`, observations and episode termination.
- Simulate random arrivals, machine capacity, deadlines and temporary failures.
- Implement at least two heuristic baselines such as FIFO and shortest-job-first.
- Implement tabular Q-learning yourself and explain every update term.
- Create low-, medium- and high-load workload generators.
- Build a visual replay showing queues, machines, decisions and reward components.
- Add DQN/PPO only as an extension after the basic system is correct.

WHAT MUST BE YOUR OWN WORK

- The environment and transition logic.
- The reward function and Q-learning implementation.
- The multi-seed experiment runner.

30-DAY BUILD PLAN

Week 1 - formal problem design, simulator and heuristic baselines.

Week 2 - Q-learning, training loop and visual episode replay.

Week 3 - reward ablations, unseen workloads and multi-seed experiments.

Week 4 - interface, documentation, tests and final policy comparison.

EVIDENCE YOU MUST SHOW

- Compare the learned policy against all heuristics on unseen workloads.
- Measure waiting time, missed deadlines, throughput, utilisation and fairness.
- Run multiple random seeds and report mean plus variation.
- Perform at least three reward-design experiments.
- Find and document any reward-hacking or unstable behaviour.

FINAL SUBMISSION PACKAGE

- Working repository with meaningful commit history.
- README with exact setup and run commands.
- Architecture diagram and important design decisions.
- Automated tests plus a written failure log.
- 5-8 minute demo video and a concise final report.
- AI_USAGE.md describing how AI tools were used and verified.

SUGGESTED TOOLS

Python, NumPy, Gymnasium-compatible interfaces and Matplotlib/Streamlit for visualisation.

HIGH BAR: Running a ready-made PPO agent on CartPole does not satisfy this project.

10 PROJECT OPTIONS: ML | DL | CV | NLP | RL | GENAI | LLM FINE-TUNING | RAG | CODING AGENT | AI EVALS

06

GENERATIVE AI / **PROMPTOPS**

Build a controlled generation platform, not a single prompt demo.

ONE-MONTH CHALLENGE: Choose exactly one of the ten projects. You have 30 days to build a working system, document your decisions and show evidence that it works. AI tools are allowed, but you must understand and be able to explain everything you submit.

WHAT YOU WILL BUILD

Build a provider-agnostic platform that converts messy instructions into reliable structured outputs. For example, an event brief could become a schedule, announcement, task list and content pack. The system must validate its output, survive model failures and make prompt/model changes measurable.

CORE REQUIREMENTS

- Create one common model interface and support at least two backends or one real plus one mock backend.
- Support streaming text and structured JSON output.
- Create a versioned prompt registry with variables, metadata and change history.
- Validate every structured response against a JSON schema.
- Add repair, retry, timeout, fallback and clear error handling.
- Add caching and track tokens, latency and estimated cost per run.
- Implement a simple routing rule based on task type, quality, speed or cost.
- Build an experiment UI to compare prompt and model versions side by side.

WHAT MUST BE YOUR OWN WORK

- The provider adapter interface and routing layer.
- The prompt/version registry and schema-validation pipeline.
- The repeatable prompt regression runner.

30-DAY BUILD PLAN

Week 1 - use case, schemas, model adapter and deterministic mock backend.

Week 2 - prompt registry, structured generation and validation/repair.

Week 3 - test suite, routing, caching, retries and telemetry.

Week 4 - experiment UI, deployment, documentation and comparison report.

EVIDENCE YOU MUST SHOW

- Create at least 50 fixed test cases with expected properties.
- Report schema-validity rate, instruction-following rate, latency, tokens and failures.
- Compare at least two prompt versions and two model configurations.
- Test malformed JSON, contradictory instructions, timeouts and interrupted streaming.
- Show examples where a cheaper/faster model is enough and where it is not.

FINAL SUBMISSION PACKAGE

- Working repository with meaningful commit history.
- README with exact setup and run commands.
- Architecture diagram and important design decisions.
- Automated tests plus a written failure log.
- 5-8 minute demo video and a concise final report.
- AI_USAGE.md describing how AI tools were used and verified.

SUGGESTED TOOLS

Python or TypeScript; any model provider or local model; FastAPI/Node and any frontend.

HIGH BAR: A form that sends one prompt to one API and prints the response is not PromptOps.

10 PROJECT OPTIONS: ML | DL | CV | NLP | RL | GENAI | LLM FINE-TUNING | RAG | CODING AGENT | AI EVALS

07

LARGE LANGUAGE MODELS / TUNEFORGE

Fine-tune an open model and prove that the tuned model is actually better.

ONE-MONTH CHALLENGE: Choose exactly one of the ten projects. You have 30 days to build a working system, document your decisions and show evidence that it works. AI tools are allowed, but you must understand and be able to explain everything you submit.

WHAT YOU WILL BUILD

Choose one narrow, useful task and fine-tune a small open-source language model for it. Possible tasks include structured extraction, code explanation, tool-call generation, technical Q&A or support response drafting. The project must cover data creation, training, evaluation, serving and honest comparison with the original model.

CORE REQUIREMENTS

- Choose a suitable open model, normally between 0.5B and 3B parameters.
- Define a clear instruction/input/output format for the chosen task.
- Create, clean, deduplicate and version a high-quality training dataset.
- Split train, validation and test data by source to reduce leakage.
- Benchmark the untouched base model before training.
- Fine-tune with LoRA or QLoRA, which trains small adapter weights instead of every model weight.
- Track training/validation loss, learning rate, gradient norm, memory use and speed.
- Serve the final model through an API and a simple comparison interface.

WHAT MUST BE YOUR OWN WORK

- The dataset preparation and validation pipeline.
- The experiment/configuration runner for repeatable training.
- The base-versus-tuned evaluation and report generator.

30-DAY BUILD PLAN

Week 1 - task design, dataset schema, cleaning and base-model benchmark.
 Week 2 - first LoRA/QLoRA run and complete training pipeline.
 Week 3 - ablations, error analysis and base-versus-tuned evaluation.
 Week 4 - inference API, comparison UI, model card and reproducible release.

EVIDENCE YOU MUST SHOW

- Evaluate the base and tuned model on the same frozen test set.
- Run at least two ablations: data size, epochs, learning rate, LoRA rank or prompt format.
- Measure task correctness, format validity, latency, memory and failure rate.
- Check whether the tuned model became worse on unrelated general prompts.
- Publish a model card describing data, limitations, risks and intended use.

FINAL SUBMISSION PACKAGE

- Working repository with meaningful commit history.
- README with exact setup and run commands.
- Architecture diagram and important design decisions.
- Automated tests plus a written failure log.
- 5-8 minute demo video and a concise final report.
- AI_USAGE.md describing how AI tools were used and verified.

SUGGESTED TOOLS

Python, Hugging Face Transformers, PEFT, TRL/Axolotl/Unsloth or equivalent. Use hardware that is available to you.

HIGH BAR: One fine-tuning run plus a few cherry-picked examples is not evidence of improvement.

10 PROJECT OPTIONS: ML | DL | CV | NLP | RL | GENAI | LLM FINE-TUNING | RAG | CODING AGENT | AI EVALS

08

RETRIEVAL-AUGMENTED GENERATION / EVIDENCEBENCH

Build a knowledge system that retrieves evidence, cites it and knows when to abstain.

ONE-MONTH CHALLENGE: Choose exactly one of the ten projects. You have 30 days to build a working system, document your decisions and show evidence that it works. AI tools are allowed, but you must understand and be able to explain everything you submit.

WHAT YOU WILL BUILD

Build a full-stack research workspace that ingests documents and answers questions using verifiable evidence. The difficult part is not the chat box; it is parsing, chunking, retrieval, reranking, citation quality, abstention, evaluation and observability. Python, TypeScript or another suitable stack is allowed.

CORE REQUIREMENTS

- Ingest PDFs, text and Markdown while preserving document, page and section metadata.
- Detect duplicate or updated documents and keep index versions.
- Implement and compare at least two chunking strategies.
- Implement keyword retrieval and dense vector retrieval.
- Combine them with a documented hybrid fusion method.
- Retrieve a wider candidate set, rerank it and expose scores in a debug view.
- Generate answers with passage-level citations and clickable source previews.
- Refuse to answer when the available evidence is weak, missing or conflicting.

WHAT MUST BE YOUR OWN WORK

- One important component such as hybrid fusion, citation alignment or chunking.
- The question-dataset runner and retrieval/answer metrics.
- The trace format that records each pipeline stage.

30-DAY BUILD PLAN

Week 1 - ingestion, metadata, baseline chunking and dense retrieval.

Week 2 - keyword/hybrid retrieval, reranking, citations and source viewer.

Week 3 - benchmark creation, metrics, abstention and observability.

Week 4 - complete UI/API, deployment, tests and final experiment report.

EVIDENCE YOU MUST SHOW

- Create 40-60 questions: direct, paraphrased, multi-document, conflicting and unanswerable.
- Measure Recall@K, MRR/nDCG, citation precision, citation coverage and abstention quality.
- Compare dense-only, keyword-only, hybrid and reranked configurations.
- Record latency, tokens, model/index version, retrieved chunks and reranker scores.
- Document at least 20 retrieval or generation failures and their causes.

FINAL SUBMISSION PACKAGE

- Working repository with meaningful commit history.
- README with exact setup and run commands.
- Architecture diagram and important design decisions.
- Automated tests plus a written failure log.
- 5-8 minute demo video and a concise final report.
- AI_USAGE.md describing how AI tools were used and verified.

SUGGESTED TOOLS

Python or TypeScript; vector database of your choice; optional LangGraph/LangChain/LlamaIndex, but core reasoning must remain visible.

HIGH BAR: PDF upload + fixed top-3 vector search + an LLM answer is only the starting baseline.

10 PROJECT OPTIONS: ML | DL | CV | NLP | RL | GENAI | LLM FINE-TUNING | RAG | CODING AGENT | AI EVALS

09

AI AGENTS / **CODEFORGE**

Build your own repository-aware coding-agent harness - not a coding chatbot.

ONE-MONTH CHALLENGE: Choose exactly one of the ten projects. You have 30 days to build a working system, document your decisions and show evidence that it works. AI tools are allowed, but you must understand and be able to explain everything you submit.

WHAT YOU WILL BUILD

Build a local coding-agent runtime in the spirit of tools such as OpenCode and pi-agent. A user gives the agent a software task and a repository. The agent must inspect the repository, plan changes, call safe tools, edit files, run tests, inspect the Git diff and decide whether the task is complete. The harness is the runtime around the model: its tools, context selection, state, safety, patching, verification, tracing and stopping rules. You may study existing open-source agents for ideas, but your harness and orchestration must be your own.

CORE REQUIREMENTS

- Create a CLI or TUI that accepts a task and repository path.
- Implement structured tools: list/search files, read ranges, create files, apply patches, run tests, Git status/diff and restore.
- Validate every tool call against a strict schema before execution.
- Build a repository map and select relevant context instead of sending the whole codebase.
- Implement an explicit loop: inspect → plan → act → observe → verify → stop/continue.
- Use targeted patches, detect conflicts and support rollback.
- Add step limits, retries, repeated-action/loop detection and resumable run state.
- Require tests, lint or another verification step before the agent can declare success.

WHAT MUST BE YOUR OWN WORK

- The agent loop/state machine and stopping rules.
- The tool runtime, permissions and safe command executor.
- The context manager, repository map and token-budget logic.

30-DAY BUILD PLAN

Week 1 - repository tools, schemas, safe execution and simple one-step edits.

Week 2 - planning loop, patching, context selection and Git/test integration.

Week 3 - recovery, loop detection, traces and the 15-task benchmark.

Week 4 - CLI/TUI polish, comparisons, documentation and final agent demonstration.

EVIDENCE YOU MUST SHOW

- Sandbox commands with working-directory limits, timeouts, output limits and environment protection.
- Save a complete run trace: prompts, tools, results, patches, commands, tests, tokens and final diff.
- Create at least 15 real tasks across two small repositories: bugs, missing features, refactors and tests.
- Measure completed tasks, passing tests, invalid tool calls, steps, changed files, latency and tokens.
- Compare two harness designs, such as no-plan vs plan or whole-repo context vs selected context.

FINAL SUBMISSION PACKAGE

- Working repository with meaningful commit history.
- README with exact setup and run commands.
- Architecture diagram and important design decisions.
- Automated tests plus a written failure log.
- 5-8 minute demo video and a concise final report.
- AI_USAGE.md describing how AI tools were used and verified.

SUGGESTED TOOLS

Python or TypeScript; any LLM API/local model. Existing coding-agent frameworks cannot be used as the core engine.

HIGH BAR: A chat interface that prints code, or a wrapper around an existing coding agent, does not satisfy this task.

10 PROJECT OPTIONS: ML | DL | CV | NLP | RL | GENAI | LLM FINE-TUNING | RAG | CODING AGENT | AI EVALS

10

AI EVALUATION / **EVALFORGE**

Build the platform that tells an AI team whether a change truly improved the system.

ONE-MONTH CHALLENGE: Choose exactly one of the ten projects. You have 30 days to build a working system, document your decisions and show evidence that it works. AI tools are allowed, but you must understand and be able to explain everything you submit.

WHAT YOU WILL BUILD

Build a framework-agnostic evaluation platform for prompts, models, RAG pipelines and agents. The platform should run versioned test datasets, collect outputs, apply several kinds of checks, compare runs and produce a report that developers can trust. It must show individual failures, not only one final score.

CORE REQUIREMENTS

- Define a versioned JSONL or database schema for test cases and expected behaviour.
- Create adapters so at least two different systems can be evaluated through one interface.
- Support synchronous and asynchronous execution with controlled concurrency.
- Add exact/rule-based checks, semantic checks and task-specific custom metrics.
- Add an optional rubric-based LLM judge and pairwise A/B comparison.
- Repeat selected tests to measure nondeterministic behaviour.
- Track model/prompt/system version, latency, tokens, cost, errors and raw outputs.
- Build a dashboard that compares runs, filters failures and replays selected cases.

WHAT MUST BE YOUR OWN WORK

- The runner, adapter/plugin interface and result store.
- The comparison/regression engine.
- The report generator and failed-case replay flow.

30-DAY BUILD PLAN

Week 1 - test schema, system adapter and basic deterministic runner.

Week 2 - result store, metrics, concurrency and run versioning.

Week 3 - LLM judge, pairwise comparison, human calibration and regression gates.

Week 4 - dashboard, replay, CI integration, documentation and final case study.

EVIDENCE YOU MUST SHOW

- Create at least 50 test cases and evaluate two genuinely different systems.
- Compare automated judge labels with a small human-labelled set.
- Analyse judge disagreement, order bias and cases where the judge is unreliable.
- Add configurable regression gates that can fail a CI command.
- Demonstrate at least three real regressions discovered by the platform.

FINAL SUBMISSION PACKAGE

- Working repository with meaningful commit history.
- README with exact setup and run commands.
- Architecture diagram and important design decisions.
- Automated tests plus a written failure log.
- 5-8 minute demo video and a concise final report.
- AI_USAGE.md describing how AI tools were used and verified.

SUGGESTED TOOLS

Python or TypeScript; database and frontend of your choice; any model provider for optional judge-based scoring.

HIGH BAR: A script that prints one average score is not an evaluation platform.